

HỌC MÁY CƠ BẢN VỚI PYTHON

Chương 4 – Buổi 2

Mã hóa, co giãn và quy trình chống rò rỉ

(Encoding, Scaling & Pipelines)

Lê Nhật Tùng

Bài giảng đi kèm giáo trình

Mục tiêu buổi học

Sau buổi học này, bạn có thể:

- **Mã hóa** (*encode*) biến phân loại: nhị phân, **thứ bậc** (*ordinal*), **one-hot** (*one-hot encoding*).
- **Co giãn đặc trưng** (*feature scaling*) bằng Min-Max và **chuẩn hóa z-score** (*standardization*); tính tay và giải thích vì sao co giãn giúp kNN.
- Nhận biết và phòng tránh **rò rỉ dữ liệu** (*data leakage*).
- Xây dựng quy trình tiền xử lý hoàn chỉnh bằng `Pipeline` và `ColumnTransformer`.

Nội dung

1. Mã hóa biến phân loại

2. Co giãn đặc trưng

3. Rò rỉ dữ liệu và Pipeline

4. Thực hành Buổi 4.2

5. Tổng kết

Phần 1

Mã hóa biến phân loại

Chuẩn bị: làm lại các bước làm sạch của Buổi 4.1

ch04.py — ô chuan_bi_buoi2

```python
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
pd.set_option("display.max_columns", None)
pd.set_option("display.width", 85)
# Làm lại các bước làm sạch của Buổi 4.1
titanic = sns.load_dataset("titanic")
df = titanic.drop(columns=["alive", "class", "who", "adult_male", "embark_town", "deck"])
df["age"] = df["age"].fillna(df["age"].median())
df["embarked"] = df["embarked"].fillna(df["embarked"].mode()[0])
print(df.shape, " số ô trống:", df.isna().sum().sum())
```

Kết quả

```
(891, 9)  số ô trống: 0
```

Bảng `df` sau khi làm sạch: 891 dòng, 9 cột, không còn ô trống. Nhưng còn **cột chữ** mà mô hình không hiểu, và các cột số có **thang đo chênh lệch**.

Biến phân loại (*categorical variables*) và mã hóa nhị phân

Định nghĩa: Hai loại biến phân loại

- **Định danh** (*nominal*): chỉ là tên gọi, **không có thứ tự** – cảng lên tàu, màu sắc, thành phố.
- **Thứ bậc** (*ordinal*): **có thứ tự** tự nhiên – hạng vé, trình độ, mức hài lòng.

ch04.py — ô ma_hoa_nhi_phan

```
1 df["sex"] = df["sex"].map({"male": 0, "female": 1})
2 print(df["sex"].value_counts())
```

Kết quả

```
sex
0    577
1    314
Name: count, dtype: int64
```

- Hai giá trị (giới tính) ⇒ 0 và 1.
- Thứ bậc ⇒ số nguyên giữ thứ tự: trung cấp = 0, cao đẳng = 1, đại học = 2. (`pclass` đã ở dạng này.)

Mã hóa thứ bậc: tự viết từ điển thứ tự

ch04.py — ô ma_hoa_thu_bac

```
trinh_do = pd.Series(["đại học", "trung cấp", "cao đẳng", "đại học", "trung cấp"])
thu_tu = {"trung cấp": 0, "cao đẳng": 1, "đại học": 2}    # giữ đúng thứ tự
print(pd.DataFrame({"trinh_do": trinh_do, "ma_so": trinh_do.map(thu_tu)}))
```

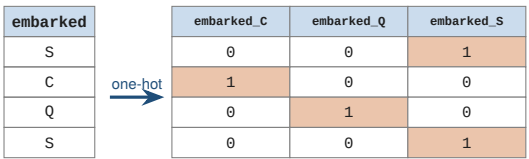
Kết quả

```
    trinh_do  ma_so
0    đại học      2
1  trung cấp      0
2   cao đẳng      1
3    đại học      2
4  trung cấp      0
```

- `map(từ_điển)`: thay mỗi giá trị theo từ điển.
- Phải **tự viết** thứ tự: máy không biết “đại học” cao hơn “cao đẳng”.
- Cột `pclass` của Titanic đã ở sẵn dạng **thứ bậc** (*ordinal*) (1, 2, 3).

Mã hóa one-hot

Mã hóa C = 0, Q = 1, S = 2 (*label encoding*) ⇒ mô hình hiểu nhầm “S lớn hơn Q”, “Q nằm giữa C và S”. Cách đúng – **mã hóa one-hot** (*one-hot encoding*): **mỗi giá trị một cột 0/1**; mỗi dòng có đúng một số 1. Các cột mới gọi là **biến giả** (*dummy variables*).



ch04.py — ô ma_hoa_onehot

```
1 vi_du = pd.DataFrame({"embarked": ["S",
  ↪  "C", "Q", "S"]})
2 print(pd.get_dummies(vi_du,
  ↪  columns=["embarked"], dtype=int))
```

Kết quả

```
   embarked_C  embarked_Q  embarked_S
0           0           0           1
1           1           0           0
2           0           1           0
3           0           0           1
```

Ví dụ đời thường và bài tập mẫu: one-hot

Ví dụ đời thường: Phiếu khảo sát có ô đánh dấu

"Bạn lên tàu ở cảng nào?" ☐ Cherbourg ☐ Queenstown ☐ Southampton. Đánh dấu đúng một ô; ghi lại bằng ba cột 0/1 chính là one-hot. Không ô nào "lớn hơn" ô nào.

Bài tập mẫu: Cột `mau`: đỏ, xanh, đỏ, vàng, xanh

`mau`	vàng	xanh	đỏ
đỏ	0	0	1
xanh	0	1	0
đỏ	0	0	1
vàng	1	0	0
xanh	0	1	0

Nếu mã hóa đỏ = 0, vàng = 1, xanh = 2:

- mô hình hiểu "xanh $= 2\times$ vàng", "vàng nằm giữa";
- với kNN: khoảng cách đỏ–xanh (2) gấp đôi đỏ–vàng (1).

Với one-hot: khoảng cách giữa hai màu khác nhau **luôn bằng** $\sqrt{2}$ – ba màu "bình đẳng".

Áp dụng cho Titanic

ch04.py — ô ma_hoa_titanic

```python
df = pd.get_dummies(df, columns=["embarked"], dtype=int)
df["alone"] = df["alone"].astype(int)
print(df.head(3))
```

Kết quả

```
   survived  pclass  sex   age  sibsp  parch     fare  alone  embarked_C  \
0         0       3    0  22.0      1      0   7.2500      0           0
1         1       1    1  38.0      1      0  71.2833      0           1
2         1       3    1  26.0      0      0   7.9250      1           0

   embarked_Q  embarked_S
0           0           1
1           0           0
2           0           1
```

Lưu ý

Cột có hàng nghìn giá trị (*high cardinality*) (mã sản phẩm, tên đường) ⇒ one-hot tạo hàng nghìn cột. Khi đó gom giá trị hiếm thành “khác” hoặc dùng kỹ thuật mã hóa khác.

Tạo đặc trưng mới (*feature engineering*): quy mô gia đình

ch04.py — ô dac_trung_moi

```
df["family_size"] = df["sibsp"] + df["parch"] + 1
print(df.groupby("family_size")["survived"].agg(
    ↪ ["count", "mean"]).round(3))
```

Kết quả

family_size	count	mean
1	537	0.304
2	161	0.553
3	102	0.578
4	29	0.724
5	15	0.200
6	22	0.136
7	12	0.333
8	6	0.000
11	7	0.000

Kỹ thuật đặc trưng (*feature engineering*): kết hợp cột có sẵn thành đặc trưng giàu thông tin hơn.

- Đi một mình: chỉ 30% sống sót.
- Gia đình 2–4 người: 55–72%.
- Từ 5 người: rất thấp (khó tập hợp cả nhà lên xuồng).

Quan hệ “tăng rồi giảm” này khó thấy khi nhìn riêng `sibsp` hay `parch`.

Phần 2

Co giãn đặc trưng

Hai cách co giãn (*feature scaling*) phổ biến

Toán dễ hiểu: Min-Max scaling: về khoảng $[0, 1]$

$$x' = \frac{x - x_{\text{min}}}{x_{\text{max}} - x_{\text{min}}}$$

Nhỏ nhất $\to$ 0, lớn nhất $\to$ 1.

Toán dễ hiểu: Chuẩn hóa z-score – standardization

$$z = \frac{x - \bar{x}}{\sigma}$$

Trung bình 0, độ lệch chuẩn 1. $z = 2$: cao hơn trung bình 2 độ lệch chuẩn.

Hai cách co giãn (*feature scaling*) phổ biến

Toán dễ hiểu: Min-Max scaling: về khoảng $[0, 1]$

$$x' = \frac{x - x_{\min}}{x_{\max} - x_{\min}}$$

Nhỏ nhất $\to$ 0, lớn nhất $\to$ 1.

Toán dễ hiểu: Chuẩn hóa z-score – standardization

$$z = \frac{x - \bar{x}}{\sigma}$$

Trung bình 0, độ lệch chuẩn 1. $z = 2$: cao hơn trung bình 2 độ lệch chuẩn.

Tính tay $x = (20, 30, 50)$: $x_{\min} = 20$, $x_{\max} = 50$, $\bar{x} = 33{,}33$, $\sigma \approx 12{,}47$.

x	Min-Max $\frac{x-20}{30}$	z-score $\frac{x-33{,}33}{12{,}47}$
20	0	$-1{,}069$
30	0,3333	$-0{,}267$
50	1	1,336

Ví dụ đời thường: so sánh điểm của hai đề thi khác nhau

Ví dụ đời thường: Ai giỏi hơn so với lớp?

An được 8 điểm Toán (đề khó: lớp trung bình 5, phần lớn từ 4 đến 6). Bình được 9 điểm Văn (đề dễ: lớp trung bình 8,5, phần lớn từ 8 đến 9).

- So điểm thô: Bình hơn ($9 > 8$).
- So với lớp: An hơn trung bình **3 điểm** khi điểm cả lớp chỉ dao động ≈ 1 điểm; Bình chỉ hơn **0,5 điểm**.
- **z-score** (*standard score*) làm đúng phép so sánh này: "cách trung bình bao nhiêu lần độ lệch chuẩn". An: $z \approx 3$; Bình: $z \approx 1$.

Bài tập mẫu: tính tay Min-Max và z-score

Cột có giá trị $(2, 4, 6, 8)$ (độ lệch chuẩn chia n).

Bài tập mẫu

Lời giải. Min-Max: min $= 2$, max $= 8$, max $-$ min $= 6 \Rightarrow 0$; $0{,}333$; $0{,}667$; 1.

z-score: $\bar{x} = 5$; độ lệch $(-3, -1, 1, 3)$; bình phương $(9, 1, 1, 9)$, trung bình $20/4 = 5 \Rightarrow \sigma = \sqrt{5} \approx 2{,}236$.
Kết quả: $-3/2{,}236 \approx -1{,}342$; $-0{,}447$; $0{,}447$; $1{,}342$.

Kiểm tra: các giá trị z đối xứng quanh 0 (trung bình 0); giá trị 8 “cách trung bình 1,34 độ lệch chuẩn”.

Kiểm tra bằng Python

ch04.py — ô co_gian_tay

```
x = np.array([20.0, 30.0, 50.0])
x_minmax = (x - x.min()) / (x.max() - x.min())
x_z = (x - x.mean()) / x.std()
print("Min-max :", x_minmax.round(4))
print("Z-score :", x_z.round(4))
print("Z-score có trung bình", round(x_z.mean(),
    ↪ 4), "và độ lệch chuẩn", round(x_z.std(), 4))
```

Kết quả

```
Min-max : [0.     0.3333 1.    ]
Z-score : [-1.069  -0.2673  1.3363]
Z-score có trung bình -0.0 và độ lệch chuẩn 1.0
```

`-0.0` chính là 0: sai số làm tròn cực nhỏ của số thực. Bên phải: `MinMaxScaler`, `StandardScaler` của `scikit-learn` cho cùng kết quả – đều là **bộ biến đổi** (*transformer*) (`fit` rồi `transform`).

ch04.py — ô co_gian_sklearn

```
from sklearn.preprocessing import MinMaxScaler,
    ↪ StandardScaler
cot_so = df[["age", "fare"]]
print(cot_so.describe().loc[["mean", "std",
    ↪ "min", "max"]].round(2))
chuan = StandardScaler().fit_transform(cot_so)
print("Sau StandardScaler: trung bình",
    ↪ chuan.mean(axis=0).round(4),
      " độ lệch chuẩn",
       ↪ chuan.std(axis=0).round(4))
```

Kết quả

```
        age    fare
mean  29.36   32.20
std   13.02   49.69
min    0.42    0.00
max   80.00  512.33
Sau StandardScaler: trung bình [0. 0.]  độ lệch
 ↪ chuẩn [1. 1.]
```

Khi có ngoại lai: Min-Max hay z-score?

ch04.py — ô co_gian_ngoai_lai

```python
x = np.array([[10.0], [20.0], [20.0], [30.0], [200.0]])   # 200 là ngoại lai
from sklearn.preprocessing import MinMaxScaler, StandardScaler
print("Min-Max :", MinMaxScaler().fit_transform(x).ravel().round(3))
print("Z-score :", StandardScaler().fit_transform(x).ravel().round(3))
```

Kết quả

```
Min-Max : [0.    0.053 0.053 0.105 1.   ]
Z-score : [-0.636 -0.498 -0.498 -0.36   1.992]
```

- Min-Max: 200 thành 1, bốn giá trị bình thường bị **ép** vào $[0;\ 0{,}105]$ – gần như không phân biệt được nhau.
- z-score: bốn giá trị bình thường trải từ $-0{,}64$ đến $-0{,}36$; 200 nằm ở 1,99. Cũng bị ảnh hưởng nhưng **ít hơn**.
- `.ravel()`: in mảng một cột thành một dòng.

Co giãn giúp kNN thế nào? Quay lại Wine

ch04.py — ô wine_co_gian

```python
from sklearn.datasets import load_wine
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier

X, y = load_wine(return_X_y=True)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42, stratify=y)

knn = KNeighborsClassifier(n_neighbors=5)
print("Không co giãn :", round(knn.fit(X_train, y_train).score(X_test,
    y_test), 4))
for bo_co_gian in [MinMaxScaler(), StandardScaler()]:
    bo_co_gian.fit(X_train)                    # CHỈ học từ tập huấn luyện
    Xtr = bo_co_gian.transform(X_train)
    Xte = bo_co_gian.transform(X_test)          # dùng lại thông số đã học
    diem = knn.fit(Xtr, y_train).score(Xte, y_test)
    print(f"{type(bo_co_gian).__name__:14s}:", round(diem, 4))
```

Kết quả

```
Không co giãn : 0.7222
MinMaxScaler  : 0.963
StandardScaler: 0.9444
```

Co giãn giúp kNN thế nào? Quay lại Wine

ch04.py — ô wine_co_gian

```python
from sklearn.datasets import load_wine
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier

X, y = load_wine(return_X_y=True)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42, stratify=y)

knn = KNeighborsClassifier(n_neighbors=5)
print("Không co giãn :", round(knn.fit(X_train, y_train).score(X_test,
    y_test), 4))
for bo_co_gian in [MinMaxScaler(), StandardScaler()]:
    bo_co_gian.fit(X_train)                    # CHỈ học từ tập huấn luyện
    Xtr = bo_co_gian.transform(X_train)
    Xte = bo_co_gian.transform(X_test)          # dùng lại thông số đã học
    diem = knn.fit(Xtr, y_train).score(Xte, y_test)
    print(f"{type(bo_co_gian).__name__:14s}:", round(diem, 4))
```

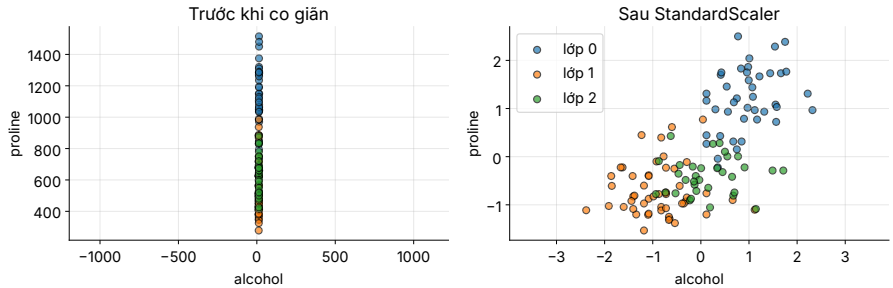
Kết quả

```
Không co giãn : 0.7222
MinMaxScaler  : 0.963
StandardScaler: 0.9444
```

Từ **72%** lên **94–96%** mà **không đổi thuật toán**.

Bộ co giãn chỉ `fit` trên `X_train`, rồi `transform` cả hai tập – tránh rò rỉ dữ liệu.

Vì sao? Nhìn bằng hình



- **Trước**: trục `proline` (hàng trăm – nghìn) dài gấp hàng trăm lần trục `alcohol` (11–15) $\Rightarrow$ mọi điểm dồn thành một đường thẳng đứng; khoảng cách gần như chỉ do `proline` quyết định.
- **Sau**: hai trục cùng thang đo, ba lớp hiện ra rõ ràng hơn.

Chọn cách co giãn

	Min-Max	**Chuẩn hóa z-score**
Kết quả	Mọi giá trị trong $[0, 1]$	Trung bình 0, độ lệch chuẩn 1
Ngoại lai *(outliers)*	Rất nhạy: một giá trị cực lớn "nén" các giá trị khác về gần 0	Ít nhạy hơn
Hay dùng cho	Ảnh (điểm ảnh 0–255), mạng nơ-ron	kNN, SVM, hồi quy có điều chuẩn, PCA

Mẹo nhỏ

Không chắc thì dùng `StandardScaler`. Các thuật toán dựa trên **cây quyết định** (*decision tree*) (Chương 10–11) **không cần** co giãn, vì chỉ so sánh lớn hơn / nhỏ hơn trong từng cột.

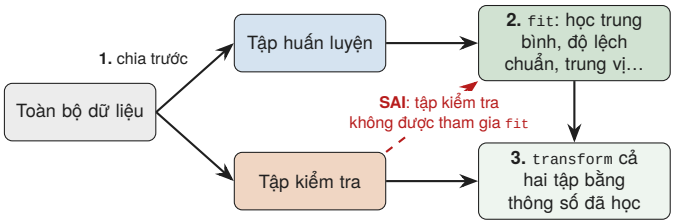
Phần 3

Rò rỉ dữ liệu và Pipeline

Rò rỉ dữ liệu

Định nghĩa: Rò rỉ dữ liệu – data leakage

Thông tin từ tập kiểm tra (hoặc thông tin không có ở thời điểm dự đoán thật) "lọt" vào huấn luyện $\Rightarrow$ kết quả đánh giá **cao giả tạo**, mô hình làm kém hơn khi dùng thật.



Giống giáo viên ra đề thi dựa trên chính bài làm của học sinh. Quy tắc: **chia tập trước**, mọi thông số tiền xử lý chỉ học từ **tập huấn luyện** (*training set*).

Ví dụ đời thường và minh họa: “lộ đề thi”

Ví dụ đời thường: Rò rỉ dữ liệu là “lộ đề” cho mô hình

Học sinh vô tình xem đề thi trước một hôm: điểm rất cao nhưng không phản ánh năng lực thật. Tập kiểm tra (đề thi) lọt vào quá trình học – dù chỉ một chút.

ch04.py — ô ro_ri_minh_hoa

```
# Thông số học được khác nhau thế nào khi fit trên toàn bộ dữ liệu và chỉ trên tập huấn luyện?
bo_sai = StandardScaler().fit(X)              # SAI: dùng cả tập kiểm tra
bo_dung = StandardScaler().fit(X_train)       # ĐÚNG: chỉ tập huấn luyện
print("Trung bình proline - toàn bộ    :", round(bo_sai.mean_[12], 2))
print("Trung bình proline - chỉ train  :", round(bo_dung.mean_[12], 2))
```

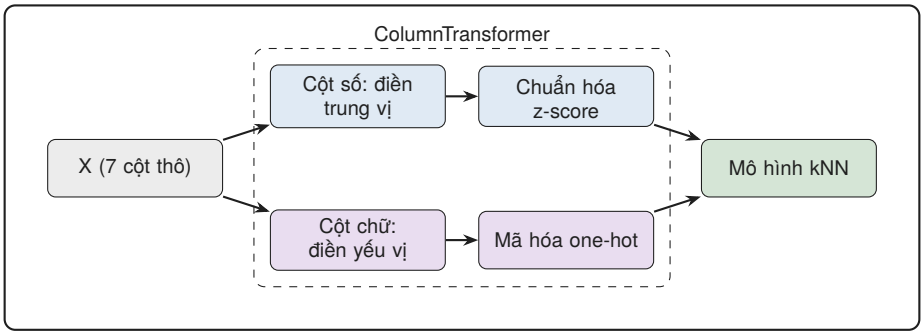
Kết quả

```
Trung bình proline - toàn bộ    : 746.89
Trung bình proline - chỉ train  : 748.33
```

Chênh nhau không nhiều (746,89 so với 748,33) nên rò rỉ ở đây “nhỏ” – nhưng nguyên tắc vẫn bị vi phạm. Với dữ liệu ít, hoặc bước tiền xử lý “học nhiều hơn” (chọn đặc trưng…), kết quả có thể đẹp giả tạo hàng chục điểm % (Chương 16).

Pipeline: gói mọi bước thành một (*pipeline*)

- `Pipeline` (*đường ống*): nối nhiều bước thành **một** mô hình; gọi `fit` một lần, mọi bước được `fit` lần lượt trên tập huấn luyện; `predict` tự áp dụng đúng các bước cho dữ liệu mới.
- `ColumnTransformer`: xử lý **khác nhau** cho nhóm cột số và nhóm cột chữ.



Pipeline

Pipeline cho Titanic: mã nguồn

ch04.py — ô pipeline_titanic

```
from sklearn.compose import ColumnTransformer
from sklearn.impute import SimpleImputer
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder

data = sns.load_dataset("titanic")
X = data[["pclass", "sex", "age", "sibsp", "parch", "fare", "embarked"]]
y = data["survived"]
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)

cot_so = ["age", "sibsp", "parch", "fare", "pclass"]
cot_chu = ["sex", "embarked"]
xu_ly_so = Pipeline([("dien_thieu", SimpleImputer(strategy="median")),
                     ("co_gian", StandardScaler())])
xu_ly_chu = Pipeline([("dien_thieu", SimpleImputer(strategy="most_frequent")),
                      ("one_hot", OneHotEncoder(handle_unknown="ignore"))])
tien_xu_ly = ColumnTransformer([("so", xu_ly_so, cot_so),
                                ("chu", xu_ly_chu, cot_chu)])
mo_hinh = Pipeline([("tien_xu_ly", tien_xu_ly),
                    ("knn", KNeighborsClassifier(n_neighbors=7))])
mo_hinh.fit(X_train, y_train)
print("Accuracy trên tập kiểm tra:", round(mo_hinh.score(X_test, y_test), 4))
```

Pipeline cho Titanic: đọc mã theo sơ đồ

Kết quả

```
Accuracy trên tập kiểm tra: 0.7989
```

- Dòng 6–10: chọn 7 cột từ dữ liệu *thô*, **chia tập ngay từ đầu** (80/20).
- Dòng 12–17: hai dây chuyền nhỏ – cột số (điền trung vị $\rightarrow$ chuẩn hóa), cột chữ (điền yếu vị $\rightarrow$ one-hot). `handle_unknown="ignore"`: không báo lỗi khi gặp giá trị lạ.
- Dòng 18–19: `ColumnTransformer` ghép hai dây chuyền, mỗi cái xử lý nhóm cột của nó.
- Dòng 20–22: `Pipeline` lớn nối tiền xử lý với kNN; chỉ **một** lệnh `fit`.

Gần **80%** – cao hơn hẳn mốc **61,6%** của việc “luôn đoán thiệt mạng”.

Dự đoán thẳng từ dữ liệu thô

ch04.py — ô pipeline_du_doan

```
khach_moi = pd.DataFrame([
    {"pclass": 1, "sex": "female", "age": 28, "sibsp": 0, "parch": 0, "fare": 80.0, "embarked": "C"},
    {"pclass": 3, "sex": "male", "age": None, "sibsp": 0, "parch": 0, "fare": 7.9, "embarked": "S"},
])
print(mo_hinh.predict(khach_moi))
```

Kết quả

```
[1 0]
```

- Khách 1 (nữ, hạng 1) ⇒ **sống sót**; khách 2 (nam, hạng 3, không rõ tuổi `None`) ⇒ **thiệt mạng**.
- Pipeline tự điền tuổi bằng trung vị đã học, tự mã hóa và co giãn trước khi đưa vào kNN.

Phần 4

Thực hành Buổi 4.2

Thực hành 4.2: Pipeline cho Penguins – Bước 1

Nạp lại dữ liệu **thô** (còn ô trống) – việc điền ô trống giờ là nhiệm vụ của pipeline. Chia tập **trước** khi tiền xử lý.

ch04.py — ô lab_buoc2

```
peng = sns.load_dataset("penguins")          # dữ liệu THÔ, chưa điền ô trống
X = peng.drop(columns="species")
y = peng["species"]
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42, stratify=y)
cot_so = ["bill_length_mm", "bill_depth_mm", "flipper_length_mm", "body_mass_g"]
cot_chu = ["island", "sex"]
print("Train:", X_train.shape, " Test:", X_test.shape)
```

Kết quả

```
Train: (258, 6)  Test: (86, 6)
```

Thực hành 4.2: Bước 2 – có và không có co giãn

ch04.py — ô lab_buoc3

```python
for co_gian in [False, True]:
    buoc_so = [("dien_thieu", SimpleImputer(strategy="median"))]
    if co_gian:
        buoc_so.append(("co_gian", StandardScaler()))
    tien_xu_ly = ColumnTransformer([
        ("so", Pipeline(buoc_so), cot_so),
        ("chu", Pipeline([("dien_thieu", SimpleImputer(strategy="most_frequent")),
                          ("one_hot", OneHotEncoder(handle_unknown="ignore"))]), cot_chu)])
    mh = Pipeline([("tien_xu_ly", tien_xu_ly), ("knn", KNeighborsClassifier(n_neighbors=5))])
    mh.fit(X_train, y_train)
    print(f"Co giãn = {co_gian!s:5s}: accuracy = {mh.score(X_test, y_test):.4f}")
```

Kết quả

```
Co giãn = False: accuracy = 0.7791
Co giãn = True : accuracy = 0.9884
```

- Không co giãn: **78%**.
- Chuẩn hóa cột số: gần **99%**.

Câu hỏi cho lớp

Trong 4 cột số, cột nào lấn át mạnh nhất khi chưa co giãn? Vì sao? (Gợi ý: bảng trung bình ở Thực hành 4.1.)

Bảng thuật ngữ Việt – Anh của buổi học

Tiếng Việt	Tiếng Anh
Biến phân loại	Categorical variable
Biến định danh	Nominal variable
Biến thứ bậc	Ordinal variable
Mã hóa nhị phân	Binary encoding
Mã hóa thứ bậc	Ordinal encoding
Mã hóa one-hot	One-hot encoding
Kỹ thuật đặc trưng	Feature engineering
Co giãn đặc trưng	Feature scaling

Tiếng Việt	Tiếng Anh
Co giãn Min-Max	Min-max scaling
Chuẩn hóa z-score	Standardization (z-score)
Rò rỉ dữ liệu	Data leakage
Đường ống xử lý	Pipeline
Biến đổi theo cột	Column transformer
Điền giá trị thiếu	Imputer
Học / biến đổi	fit / transform
Dữ liệu thô	Raw data

Phần 5

Tổng kết

Lỗi thường gặp

1. **Để cột chứa đáp án** trong đặc trưng (`alive`): dấu hiệu là accuracy gần 100%.
2. **Tiền xử lý trên toàn bộ dữ liệu rồi mới chia tập**: rò rỉ dữ liệu.
3. **Gọi `fit_transform` trên tập kiểm tra**: chỉ được `transform`.
4. **Mã hóa biến định danh thành 0, 1, 2**: mô hình hiểu sai thứ tự $\Rightarrow$ one-hot.
5. **Xóa ngoại lai máy móc**: có thể là thông tin quý (gian lận, khách VIP).
6. **Dữ liệu mới không qua đúng các bước tiền xử lý** $\Rightarrow$ dùng `Pipeline`.

Tóm tắt Chương 4

- **EDA**: 5 câu hỏi; công cụ `info`, `describe`, `isna().sum()`, `groupby`, histogram, boxplot.
- **Làm sạch**: loại cột chứa đáp án / trùng / thiếu quá nhiều; điền trung vị, yếu vị hoặc theo nhóm; ngoại lai xử lý thận trọng (logarit).
- **Mã hóa**: nhị phân $\rightarrow$ 0/1; thứ bậc $\rightarrow$ số nguyên theo thứ tự; định danh $\rightarrow$ one-hot.
- **Co giãn**: Min-Max về $[0, 1]$ hoặc z-score; rất quan trọng với kNN.
- **Chống rò rỉ**: chia tập trước, chỉ `fit` trên tập huấn luyện; `Pipeline` + `ColumnTransformer` làm tự động.

Câu hỏi thảo luận

1. Phân biệt biến định danh và biến thứ bậc; cho mỗi loại hai ví dụ và cách mã hóa.
2. Cột $(2, 4, 6, 8)$: tính tay Min-Max và z-score.
3. Vì sao kNN cần co giãn đặc trưng còn cây quyết định thì không?
4. Rò rỉ dữ liệu là gì? Nêu hai cách rò rỉ có thể xảy ra trong tiền xử lý.
5. `Pipeline` và `ColumnTransformer` giải quyết những vấn đề gì?

Bài tập Buổi 4.2

- ⋆ **4.6**: tips: one-hot cột `day`, nhị phân cột `smoker`; in 5 dòng đầu.
- ⋆ **4.7**: Cột $(10, 20, 20, 30, 70)$: tính tay Min-Max, z-score; kiểm tra bằng `MinMaxScaler`, `StandardScaler`.
- ⋆ **4.8**: `size_ao` (S, M, L, XL…): định danh hay thứ bậc? Mã hóa đúng; one-hot tay cột `thanh_pho`.
- ⋆⋆ **4.9**: Thêm `family_size` vào pipeline Titanic; vì sao không gây rò rỉ?
- ⋆⋆ **4.10**: Pipeline Titanic: thử `MinMaxScaler` và $k = 3, 5, \ldots, 15$; chọn cấu hình tốt nhất.
- ⋆⋆ **4.11**: Cố tình làm **sai** (`fit` scaler trên toàn bộ Wine); so với cách đúng qua 20 `random_state`.
- ⋆⋆⋆ **4.12**: Tự viết lớp `BoChuanHoa` (`fit`, `transform`) bằng NumPy; so với `StandardScaler`.

Chi tiết: *Bài tập Buổi 4.2*, Chương 4. Buổi sau: **Phần II – Chương 5: Hồi quy tuyến tính**.